FACULDADE VINCIT

Organização e Manipulação de Estrutura de Dados

7. Merge Sort

Merge Sort

FACULDADE VINCIT

Assim como o algoritmo Quicksort, o Mergesort é também utiliza a estratégia de dividir para conquistar, sendo então, um algoritmo de ordenação por intercalação ou segmentação.

Da mesma forma que o Quicksort, pressupõe-se que o Mergesort não saiba resolver a ordenação completa do arranjo, sendo um problema muito complexo. Dessa forma, o arranjo é dividido - fisicamente - em partes menores, dando origem a novos subarranjos, até que em cada arranjo exista apenas 1 ou dois elementos.

Nessa estratégia, não existe um um pivô, apenas o arranjo original é segmentado em duas outras partes com quantidades iguais - ou quase iguais, em caso de número ímpar -, até que encontre a ordem de dois ou três elementos, sendo então este o caso base ou trivial.

O Merge Sort foi criado pelo matemático e cientista da computação John Von Neumann - o mesmo da Arquitetura de Von Neumann - em 1945.

Merge Sort

FACULDADE VINCIT

A complexidade do Quicksort no **pior caso** é:

$$O(n \log n)$$

No **melhor caso** é:

$$O(n \log n)$$

E no **caso médio**:

$$O(n \log n)$$

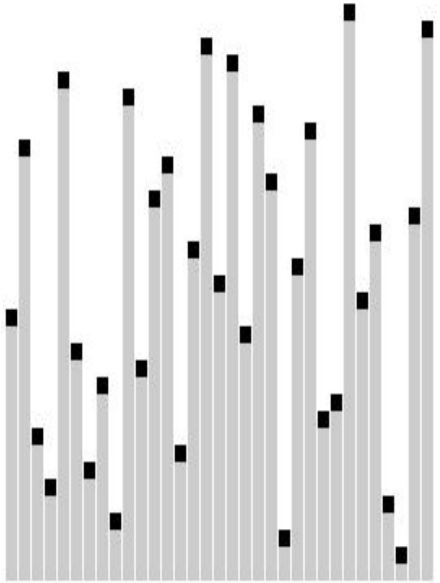


Imagem 7.1 - Ordenação por Merge Sort animado.
Fonte: akiradev.netlify.app.

Merge Sort - Funcionamento

A ordenação por Merge Sort pode ser representada por:

```
Input: Vetor V a ser ordenado
Output: Novo vetor R ordenado
begin
        if (TAMANHO(V) > 1) then
                meio ← TAMANHO(V)/2 + TAMANHO(V) MOD 2

                A ← V[1:meio]
                B ← V[meio+1:TAMANHO(V)]
                A ← MergeSort(A, TAMANHO(A))
                B ← MergeSort(B, TAMANHO(B))
                R ← Merge(A, B)
                return R
        endif
        return V
end
```

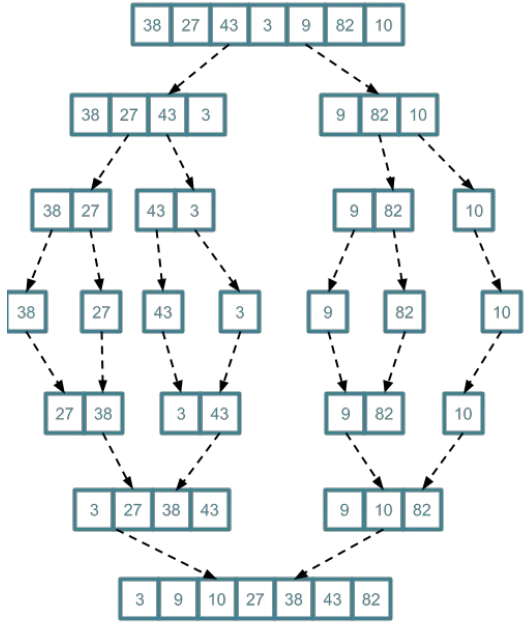


Imagem 7.2 - Sequência de ordenação por Merge Sort.
Fonte: Laureano (2008) - adaptado.

Merge Sort - Funcionamento

O processo de mesclagem pode ser entendido como:

```
Input: Vetores A e B
Output: Novo vetor M mesclado
begin
        tamanho_M ← TAMANHO(A) + TAMANHO(B)
        index_A, index_B ← 1, 1
        for i = 0 to tamanho_M do
                if (index_A > tamanhoA) then
                        Atribui B[index_B] em M e incrementa index_B
                else if (index_B > tamanho_B) then
                        Atribui A[index_A] em M e incrementa index_A
                else if
                        if (A[index_A] < B[index_B]) then
                                Atribui A[index_A] em M e incrementa index_A
                        else
                                Atribui B[index_B] em M e incrementa index_B
                        endif
                        end if
        endfor
        return M
end
```

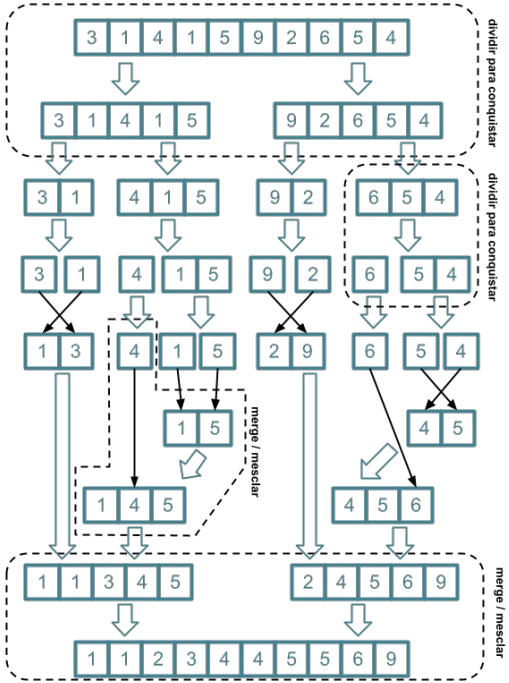


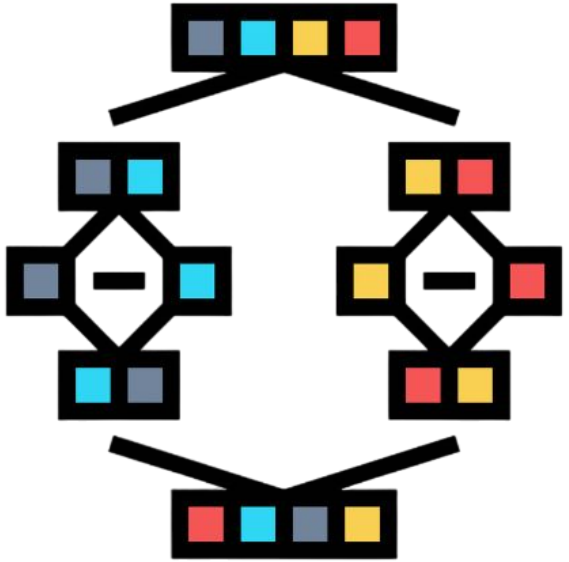
Imagem 7.3 - Processo de Mesclagem do Merge Sort.
Fonte: Laureano (2008) - adaptado.

Merge Sort - Aplicações

FACULDADE VINCIT

O Quicksort é aplicado, na maioria das vezes, para os seguintes cenários:

- **Ordenação de Listas Grandes**: Ideal para ordenar grandes volumes de dados devido à sua eficiência e estabilidade.
- **Sistemas de Arquivos**: Para ordenação de arquivos muito grandes que não cabem na memória principal (ordenando em discos).
- **Aplicações de Mídia Digital**: Ordenação de grandes coleções de dados de mídia, como imagens e vídeos.



FACULDADE VINCIT

fb.com/faculdade.vincit

linkedin.com/school/faculdade_vincit

instagram.com/faculdade_vincit